

Take-home exercises

Take-home exercises for the Forward Deployed Engineer role. Use an AI-enabled IDE throughout, and share your entire workspace in a public GitHub repository.

24h budget

3 exercises

AI-first workflow

1 Workspace rule

Start a **new, empty workspace**. Do not reuse an existing project, dotfiles directory, or personal workspace.

If you have pre-existing artifacts you want to use (skills, rules, agents, commands, prompts, MCP configs, hooks, scripts), copy them into this workspace. No symlinks. No outside references.

2 AI tool requirements

Use AI assistance throughout. Pick one primary AI tool. Supporting tools alongside are fine. Name the primary.

CATEGORY	EXAMPLES
AI coding assistant (IDE)	Cursor, GitHub Copilot, Windsurf, or similar
AI CLI	Claude Code, Codex CLI, Gemini CLI, or similar

Web-based chat interfaces (ChatGPT, Claude.ai, Gemini chat, Copilot Chat) are **not permitted as primary**.

AI-native frameworks (LangChain, LlamaIndex, Copilot Studio) are fine inside your solution. Orchestrate them through your IDE or CLI, not through their web UIs.

3 Submission

Submit a link to a **public GitHub repository**. Single repo, three exercises, full workspace. No zips, no private links.

4 Data

You generate your own dataset. Rough guidance:

- A small organization context of your invention (any industry, any size)
- Roughly 10-30 documents total, more if the problem demands it
- Variety in authors, topics, priorities, and quality

Commit the data into the repo.

Avoid real customer data, real names, or anything proprietary from current or former employers.

THE EXERCISES

EXERCISE 1

Meeting transcripts: ingestion, enrichment, query API

Single data type: meeting transcripts. Generate a corpus of meeting transcripts for an imagined organization (see Data). Markdown is fine. Each transcript includes attendees, date, and free-form conversational text with decisions and action items embedded.

Build a service that:

1. Ingests the transcript folder
2. Derives structured metadata per transcript (topic domain, priority where applicable, attendees preserved from source)
3. Stores results in a structured, queryable format
4. Exposes a query API that accepts a natural language query and returns relevant results with derived metadata

DELIVERABLE

- **Working service.** Runnable locally, callable via curl or a test script.

EXERCISE 2

Extend with Office documents: traceability and routing

Extend the system to ingest **Office documents** alongside the meeting transcripts from Exercise 1: .doc / .docx , .ppt / .pptx , .xls / .xlsx . Generate a corpus of these in the same imagined organization. Both sources queryable through a single API. Business rules apply consistently.

Every piece of information surfaced (query response, citation, routing suggestion) must trace back to its source file and author or attendees.

When the system cannot answer confidently, use traceability to suggest a routing: who to ask, why (what content matched), and a drafted question for that person.

When an answer is corrected or rejected by the consumer, capture the correction with the original query. Make gaps and corrections retrievable through the API.

Instrument the system so you would catch quality degradation before users notice.

DELIVERABLE

- **Working system.** Both sources, single API, traceability live, routing functional, gap capture operational.

EXERCISE 3

User-facing application

Build a web application that puts the full system in front of a user. The user asks a question in plain language and gets an answer with visible citations showing source file and author per claim. Derived metadata appears alongside results.

When the system cannot answer confidently, the interface presents the suggested routing (who, why, draft question) ready for the user to review, edit, send.

Gaps and corrections captured through the interface appear in a simple review queue a team lead could check periodically.

DELIVERABLES

- **Working application.** Runnable locally. React preferred, not required.
- **Measurement approach.** One metric you would track for the first 30 days post-launch, and how you would measure it. One paragraph max.

